

# 51. Découper son programme

Le même travail, écrit pour être relu dans trois jours.

## 1. Ce qu'il faut comprendre

Tout ce que fait ce programme tiendrait dans `main()`. Ce serait plus court à taper — et illisible dès qu'il grandit. Une boucle de jeu de deux cents lignes, on ne la relit pas : on la contourne, on lui ajoute une rustine à côté, et le désordre s'installe.

**Une fonction est d'abord un nom.** `dessinerLeSol()` dit ce qu'elle fait ; le lecteur n'a pas besoin d'ouvrir la boucle qui pose vingt tuiles pour le savoir. Une fonction sans argument ni retour reste utile — c'est même là qu'elle sert le plus.

**Un nombre écrit en clair au milieu du code ne dit rien.** `poser(colonne, 14, HERBE)` : pourquoi 14 ? `const uint8_t LIGNE_DU_SOL = 14;` répond, et le jour où le sol descend, il n'y a qu'un seul endroit à changer. Une `const` ne coûte **aucun octet** : sa valeur est connue à la compilation.

`return;` **au milieu d'une fonction** évite d'imbriquer trois `if`. `fairePasserLeTemps()` s'arrête tout de suite quand ce n'est pas encore l'heure, et la suite se lit à plat.

**Un programme peut tenir sur plusieurs fichiers.** `#include "dessins.cpp"` verse un autre fichier à cet endroit, avant toute compilation — et une faute y est signalée **avec le nom de son vrai fichier**, pas avec la ligne du texte collé. C'est ainsi qu'est écrit `exemples/decoupe/` : cinq fichiers, une seule cartouche, au dernier octet près.

Dans l'atelier, les fichiers sont les **onglets sous le titre** : « + fichier » en ajoute un, et il suffit de l'inclure. L'exemple « Un programme sur CINQ fichiers » les ouvre tous les cinq d'un coup, pour voir à quoi ça ressemble.

## 2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1556 octets** de cartouche.

### 1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
/*
 * Le même décor, écrit pour être RELU.
 *
 * Tout ce qui suit tiendrait dans main(). Ce serait plus court à taper, et
 * illisible trois jours plus tard.
 */
```

### 2. UNE CONSTANTE — LIGNE\_DU\_SOL

```
const uint8_t LIGNE_DU_SOL = 14;
```

### 3. UNE CONSTANTE — LIGNE\_DU\_CIEL

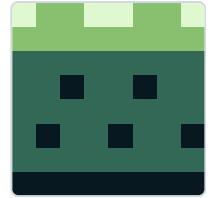
```
const uint8_t LIGNE_DU_CIEL = 3;
```

### 4. UNE CONSTANTE — COLONNES

```
const uint8_t COLONNES = 20;
```

## 5. UN DESSIN — HERBE

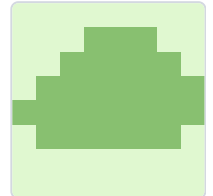
```
Tuile HERBE = {  
    "01100110",  
    "11111111",  
    "22222222",  
    "22322322",  
    "22222222",  
    "23223223",  
    "22222222",  
    "33333333",  
};
```



à l'écran

## 6. UN DESSIN — NUAGE

```
Tuile NUAGE = {  
    "00000000",  
    "00011100",  
    "00111110",  
    "01111111",  
    "11111111",  
    "01111110",  
    "00000000",  
    "00000000",  
};
```



à l'écran

## 7. UNE VARIABLE — NUAGE

```
uint8_t nuage = 3;
```

## 8. UNE VARIABLE — ATTENTE

```
uint8_t attente = 0;  
  
/* Chaque fonction porte un nom qui DIT ce qu'elle fait. */
```

## 9. UNE FONCTION — ECRIRELETITRE()

```
void ecrireLeTitre() {  
    texte(5, 1, "MON DECOR");  
}
```

## 10. UNE FONCTION — DESSINERLESOL()

```
void dessinerLeSol() {  
    for (uint8_t colonne = 0; colonne < COLONNES; colonne++) {  
        poser(colonne, LIGNE_DU_SOL, HERBE);  
    }  
}
```

## 11. UNE FONCTION — EFFACERLECIEL()

```
void effacerLeCiel() {  
    for (uint8_t colonne = 0; colonne < COLONNES; colonne++) {  
        poser(colonne, LIGNE_DU_CIEL, 0);  
    }  
}
```

## 12. UNE FONCTION — DESSINERLECIEL()

```
void dessinerLeCiel() {  
    effacerLeCiel();  
    poser(nuage, LIGNE_DU_CIEL, NUAGE);  
}
```

## 13. UNE FONCTION — FAIREPASSERLETEMPS()

```
void fairePasserLeTemps() {  
    attente++;  
    if (attente < 20) return;  
  
    attente = 0;  
    nuage++;  
    if (nuage >= COLONNES) nuage = 0;  
    dessinerLeCiel();  
}
```

## 14. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {  
    ecrireLeTitre();  
    dessinerLeSol();  
    dessinerLeCiel();  
  
    while (true) {  
        image();  
        fairePasserLeTemps();  
    }  
  
    return 0;  
}
```

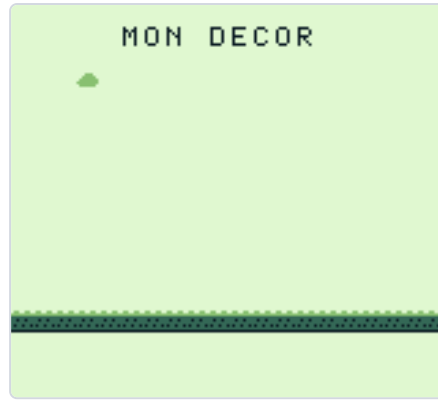
## 3. Ce qui se passe, étape par étape

---

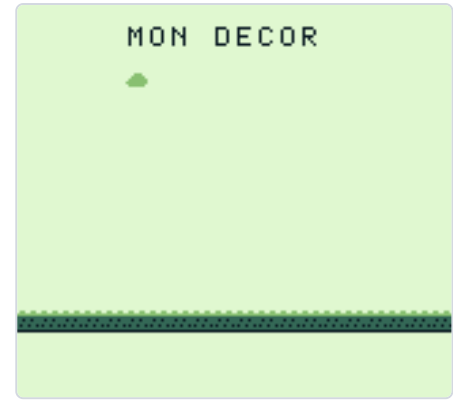
La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



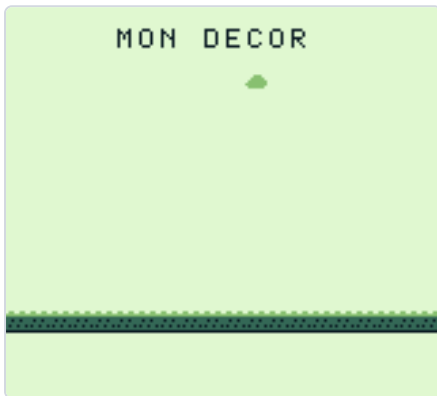
la toute première image — la console vient de démarrer — image 1



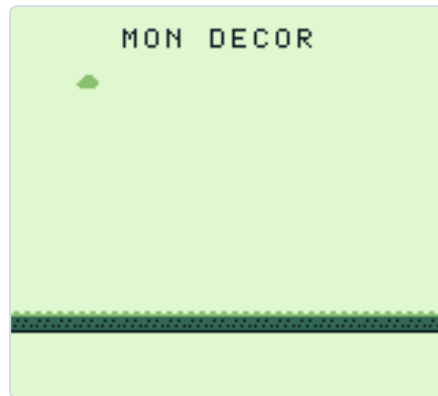
un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



au bout de trois secondes — image 180



au bout de sept secondes — image 420

## 4. Ce qu'on doit voir

Un titre, un sol d'herbe sur toute la largeur, et un nuage qui traverse le ciel.

## 5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ le sol couvre les vingt colonnes
- ✓ le titre est écrit
- ✓ un nuage est en l'air, et un seul (colonne 3)
- ✓ et il avance tout seul (de 3 à 5)

## 6. La même chose, à la souris

Cette leçon porte son atelier : le geste réécrit le programme ci-dessus, et la cartouche se refait dans la seconde. C'est le même texte, écrit autrement.

HERBE

NUAGE

+ tuile 8 × 8

+ perso 16 × 16

Game Boy — 4 nuances

Les quatre nuances de la console d'origine, et rien de plus.

Passer en couleur

nuance 3

Nuances

HERBE

tuile n° 44 — poser(x, y, HERB

- zoom

22×

+ zoom

Plein écran

Peindre ✓

Sélectionner et déplacer

Copier le dessin

Coller ici

Dupliquer

Effacer la tuile

taille réelle — 8 × 8 pixels

Ensuite : **52. Nommer un geste qu'on refait** — niveau 7 sur 10. · gameboy3 — écrire une cartouche Game Boy en C++